Deparment of Electrical, Computer and Telecommunications Engineering

Botswana International University of Science and Technology

Digital Signal Processing(Code: EEEN 414)
Assignment 1: Filter Design and Simulation

Issued Date
3 September 2024

Duration:2 weeks | *Semester 1* | Maximum Marks: 100

1. **Objectives of this Lab**
   This lab

   - Introduces you to filter design and testing techniques using MATLAB/Simulink;
   - Gives you a deeper understanding of the filter design techniques automated in MATLAB via the `Filter Design & Analysis Tool`(`FDAT`); NB: This tool will change to `filterDesigner` tool in future MATLAB Releases

2. **Prelab**
   Prior to beginning this lab, you must have carefully read over this lab in order to plan how to implement the tasks assigned. Make sure you are aware of all the items you need to include in your lab report.

3. **Grading and Due Date**

   - You should have enough time to do the lab in class, but if you do not, please note that you can come in on your own time to complete the lab requirements.
   - The lab will be graded on correctness, comprehensiveness and the insight you are able to provide when answering the questions.
   - For full points, lab reports should be written in complete sentences with correct grammar.
   - Please note STRICT DEADLINE for report on the course Teams site.

4. **Part 1: Filter Design**
   In this lab we will design digital frequency-selective filters to be used to remove or filter out noise from a signal. Recall the concept of the ideal lowpass filter, which is simply visualized in the frequency domain as the rectangle function (of a specified width and centered at the origin). Also recall that this filter, although handy in theory, is not implemented in practice. There are several reasons why ideal (lowpass, bandpass, highpass, and band-stop) filters are not used in real-life:

   1. The **impulse response,** $h[n]$ is non-causal as a consequence of the Paley-Wiener theorem, which implies these filters cannot be implemented in practice on a DSP. Another way to think about this is that $h[n]$ has infinite support for any of the ideal filters. Thus, it cannot be zero for $n < 0$ making it necessarily non-causal.
   2. When implemented in software or hardware, due to the finite number of elements employed for processing, an ideal filter exhibits the undesirable Gibbs phenomenon. In signal and

Question 4 continues on the next page...

image processing, this shows up as the infamous *ringing* effect, i.e. it introduces extra unwanted artifacts.

To bypass the above inadequacies, the design of digital filters using the **windowing technique** is an alternative to ideal filters. These filters can be implemented as FIR filters, and make use of the well-known Bartlett, Blackman, Hamming, Hanning, and Kaiser windows.

The shortcoming of digital filter design via windowing techniques is that one cannot individually control the design parameters of the filter. For example, in Figure 2, ω_p (pass-band frequency) and ω_s (stop-band frequency) cannot be independently controlled using the windowing technique.

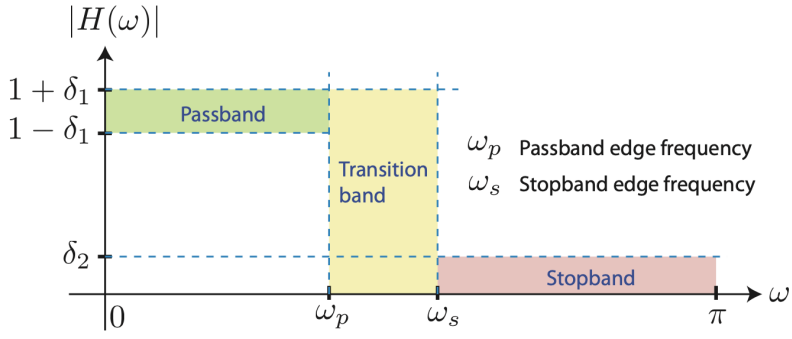

Figure 1: Filter Specification.

By making the transition band (frequencies between ω_p and ω_s) narrower, you must make a sacrifice in the form of larger undesirable ripples in the pass-band (frequencies between 0 and ω_p) and stop-band (frequencies exceeding ω_s); that is smaller $\omega_s - \omega_p$ necessitates δ_1 and δ_2 larger. These tradeoffs are all due to bypassing the two inadequacies of ideal filters.

The above specifications are general and do not delineate the form of the digital filter. In practice we would prefer either a rational system function corresponding to FIR or IIR filters discussed in the lectures. Thus we consider using the **difference equation**, or autoregressive moving average (**ARMA**) model given in equations (1) and (2) representing the descriptions in the time-and frequency-domain, respectively.

$$y[n] = -\sum_{k=1}^{N} a_k y[n-k] + \sum_{k=0}^{M} b_k x[n-k] \tag{1}$$

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{\sum_{k=1}^{N} a_k^{-k}} \tag{2}$$

Also IIR filters are generally less complex than FIR filters (the difference being that for FIR filters there is a restriction that $a_k = 0$ for all k), as they require fewer parameters and less memory for the same "quality" of filter performance. IIR filters can often be defined analytically as a rational function in the z-transform domain, and the general description of the filter in Figure 2 is not z-transform friendly (yet) - it's analog. Fortunately one can design a filter meeting Figure 2 specifications, and then transform the resulting filter to a z-transform-friendly filter by using popular transformation techniques such as approximation of derivatives,

impulse response, bilinear transformation (most popular), and matched z-transform techniques. Luckily, MATLAB has a nice little GUI driven filter design program, which requires we to do no more than click a few buttons ...

5. **Design and Implementation: FDA Tool**

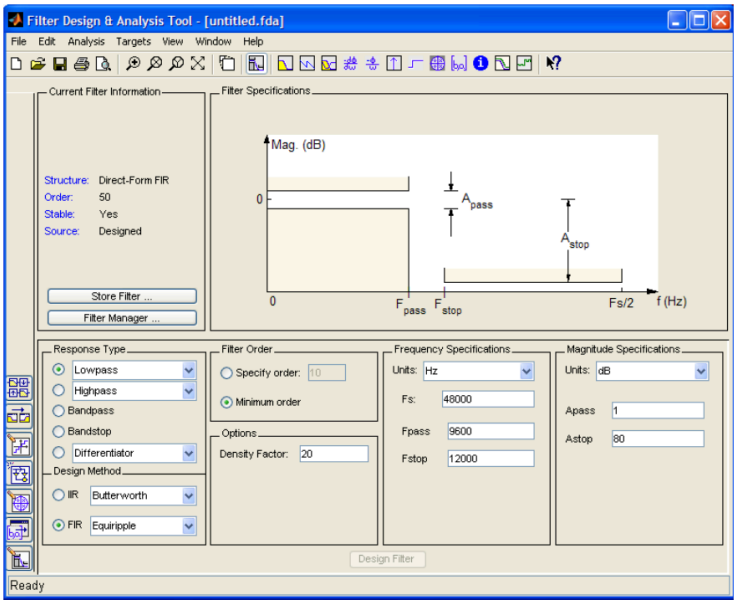

Figure 2: Filter Design and Analysis Tool.

In this section, you will learn how to use MATLAB's handy `Filter Design & Analysis Tool` (`FDAT`). To begin, start MATLAB. Now, type `fdatool` into the command window and press enter. This should, when executed, bring up the FDAT's graphical user interface (GUI), shown in Figure 2 below. The process of designing a filter is fairly self-explanatory: you simply set all of the filter specifications in the lower half of the GUI. When you are satisfied with your specifications, click on the `Design Filter` button. The magnitude response of the resulting filter will appear in the `Magnitude Response` pane. Note that you can view the coefficients of the filter's transfer function in second order sections by clicking on the Filter coefficients button at the top of the GUI (which looks like `[b,a]`).

One useful feature of FDAT is that you can store multiple filters at once and switch between them as you wish. After you have designed a filter, you can store it by selecting the `Store Filter` button; this will prompt you to enter a name for the filter. Once you have stored the filter, you can begin designing a new filter by choosing new filter specifications. To access previously stored filters, click on the `Filter Manager` button. Finally, you can save all of your stored filters in one "session" by selecting File → Save Session As; this will save your session with a `.fda` extension. You can always open previously saved sessions in the FDAT GUI.

Another cool feature of FDAT that we will be using is to export filters to a **Simulink model** as a single-input, single-output block. To do this, make sure the filter you want is currently

shown in the GUI (if not, switch to it using the `Filter Manager`). Then, click on File → Export to Simulink Model; a new set of options should appear in the lower half of the GUI. Give the filter a good, descriptive `Block Name` and make sure the Destination is Current. Press the `Realize Model Button` (this only works if you have a Simulink Model currently open, of course).

Designing the Filters

Before implementing anything, a good engineer always makes sure not to bypass the design stage. In `FDAT`, perform the following tasks, assuming a sampling rate of 8 kHz:

1. Design a minimum order, stable[1] (check if the filter is stable using information 'i' button), lowpass `Butterworth` filter with a passband frequency of 1 kHz and a stopband frequency of 1.4 kHz. Make the attenuation 1 dB at the passband frequency and 80 dB at the stopband frequency.
2. Design a minimum order, stable, lowpass `Chebyshev Type I` filter with the same specifications as the `Butterworth` filter.
3. Design a lowpass FIR filter using the `Blackman Window` with a cutoff frequency of 1 kHz. Specify the order of the filter such that the first minimum in the stopband (preceding the first lobe) is as close to 1.4 kHz as possible without exceeding it.

Discussion Questions

1. What is the order of the lowpass `Butterworth` filter you designed?
2. What is the order of the lowpass `Chebyshev Type I` filter you designed?
3. Compare the memory usage of each of the 3 filters assuming a **Direct Form II** realization is employed. Do you see how inefficient the windowing technique is? How much more expensive in terms of memory is the windowing technique from the best IIR filter?

Designing the Filters in Simulink

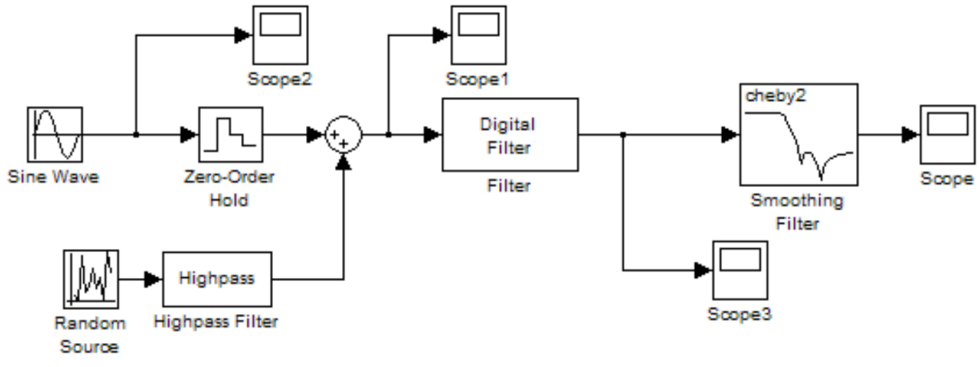

Figure 3: Filter Testing in Simulink.

In this section, you'll simulate each one of the filters you designed for a finite duration in non-real-time. Open a new Simulink model and construct a block diagram similar to that shown in Figure 3. Before your begin, please note the following about the block diagram:

[1] You can verify that a filter is stable after you have designed it by clicking on the Filter information button

(a) The `Digital Filter` block labeled "Filter" is the result of exporting a filter from `FDAT` to Simulink; this is where you'll place the filters you designed.

(b) The input to the filter is a noisy sinusoid. The high frequency noise is created in the same way as in Assignment 1. That is, the random number generator must generate numbers between -5 and 5. The highpass filter block should have an impulse response that is IIR, a minimum order, frequency in units of Hz, a F_s of 8000, F_{stop} of 3600, F_{pass} of 3800, A_{stop} of 60 and A_{pass} of 1. It may make things easier to bring a hardcopy of the Assignment 1 instructions to this lab. Please also note that the highpass filter will have to be appropriately replaced to generate different types of noise in this lab.

(c) The `Sine Wave` source block is a continuous-time source. The `Zero-Order Hold` block is used to convert it to discrete-time and thus acts like an analog-to-digital converter. Make sure that the zero-order hold is set to the same sampling rate that your filters were designed for.

(d) The `Smoothing Filter` is just a lowpass analog filter that interpolates the digital output of the `Digital Filter` and thus acts like a digital-to-analog converter. You can obtain the `Analog Filter Design` block. Recall from DSP theory that the cutoff frequency of this filter should be half of the sampling frequency. Also, note that you can choose whatever type of analog filter you wish (i.e. it doesn't have to be a Chebyshev Type II filter, as shown in the Figure). Finally, you'll want to choose a reasonably low filter order (like 4) because an analog filter tends to be the computational bottleneck that impedes the real-time simulation greatly.

(e) For any scope output, to ensure that a greater number of points are stored and hence displayed, on the scope window go to Parameters → Data History. Enter an appropriate number for the "Limit data points to last" field.

In Simulink, go to Simulation → Configuration Parameters. Please keep in mind to be sure that the solver type is variable-step. Also, choose `ode45` for the solver.

Test each of the three filters you designed in the block diagram of Figure 3 (to change filters, just delete the old one and import a different one from `FDAT`). Specifically, perform the following tasks for each filter, noting the various filter responses in your report. For every case, save and insert the oscilloscope displays on your report. Please provide information on the specific frequencies used for the sinusoidal source and the filter parameters used for generating the noise. Write down any observation that you may have about the signals and interpret the obtained results.

(a) Generate and test sine waves with frequencies in the following regions. For each frequency, set the stop time of the simulation so that you can see several periods of the sine wave. For these tests, you can disconnect the noise from the input signal and terminate it with a Terminator block.

   i. Passband region
   ii. Transition region
   iii. Stopband region

(b) Reconnect the noise to the input signal. Change the frequency of the sine wave to be in the passband and set the stop time of the simulation appropriately. Add noise to the sine wave whose frequency is in the following three regions (Note: in order to contain the noise to the correct region, you'll have to change the filter after the noise generator to the appropriate highpass, lowpass, or bandpass filter):

 Question 5 continues on the next page...

i. Passband region
ii. Transition region
iii. Stopband region

(c) To measure the effectiveness of your noise filtering approach, using basic Simulink blocks, find a way to calculate the mean squared error between the input sine wave (without noise) and the output of the overall system (please note that if your filter shifts the sine wave so it has a different shift/phase compared to the original source, then you must reshift it back before you compute this error so that both the original and filtered sinusoids are aligned). Please show how you implement this in your report; you may need a different shift for each of your filters. Discuss the performance of each filter for the various input sinusoid and noise considered. Explain in what situations you have the best/worst mean square error and why.
Therefore, to test each of your filters, please make sure that the sine wave is set to a frequency in the passband. Then, add noise to the sine wave whose frequency is in the following three regions: passband, transition and stopband. Run a simulation for each type of noise. Repeat this for all three filters you designed. For every case, save and insert the oscilloscope displays on your report. Write down any observation that you may have about the signals and interpret the your results.

End of Questions